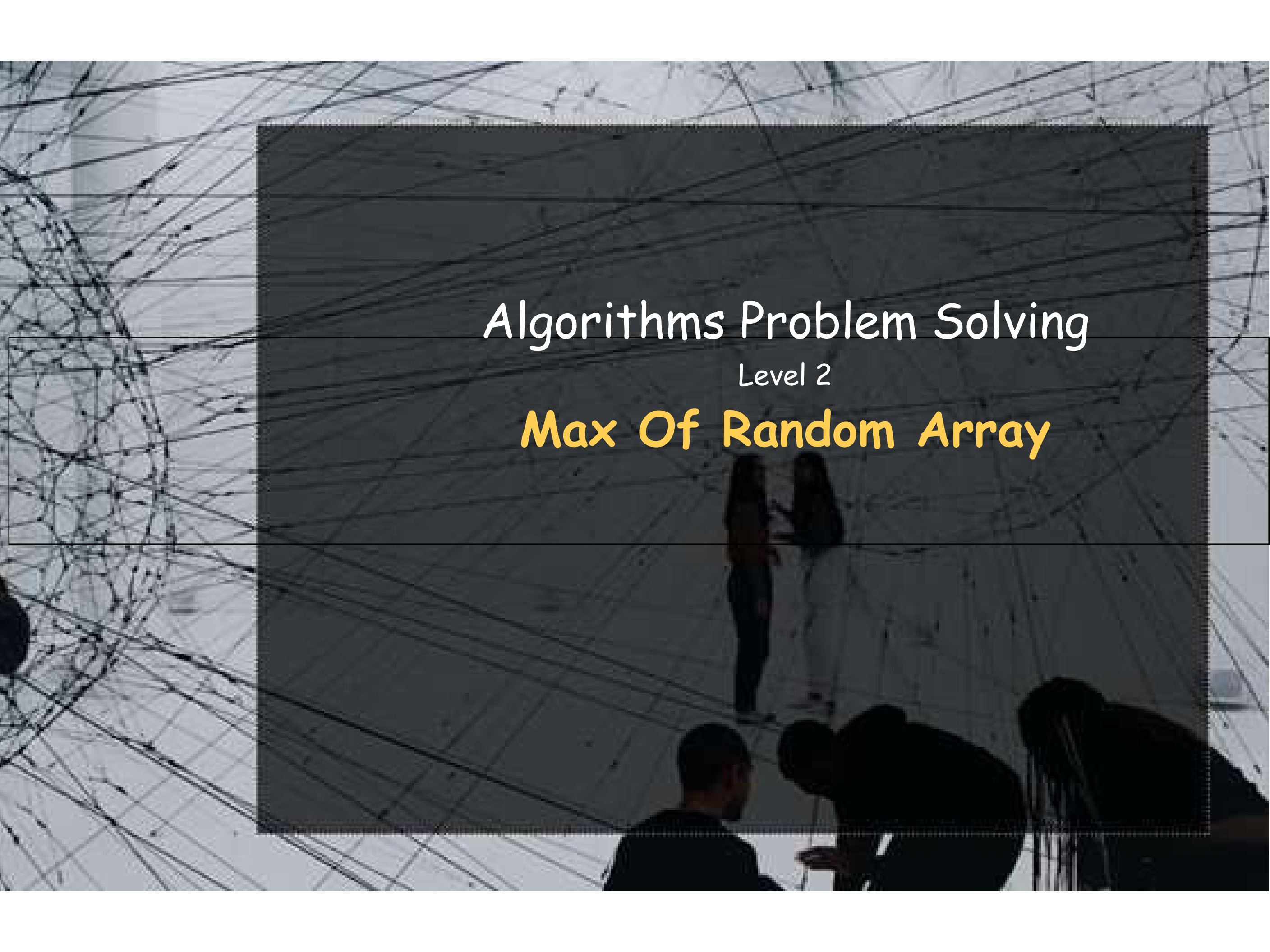


# Algorithms Problem Solving

## Level 2

### Max Of Random Array

The background of the slide is a photograph of a modern building's interior. It features a large, curved glass ceiling that allows natural light to filter through, creating a pattern of light and shadow on the floor. In the lower part of the image, the silhouettes of several people are visible, some standing and some sitting, suggesting a collaborative workspace or a public area. The overall color palette is muted, with greys, blues, and the warm tones of the light coming through the glass.

# Algorithms Problem Solving

Level 2

## Max Of Random Array

## Definition

- ◆ A program that fills an array with random numbers from `1` to `100`, prints the array, and finds the maximum number stored in it.
- ◆ The program uses a loop to compare all array elements and determine the largest value.

## Explanation

- ◆ ``FillArrayWithRandomNumber()`` asks the user for the number of elements and fills the array with random numbers from ``1`` to ``100``.
- ◆ ``PrintArrayElements()`` prints all the elements stored in the array.
- ◆ ``MaxNumbInArray()`` searches for the largest value in the array.
- ◆ The variable ``max`` starts at ``0``.
- ◆ Each array element is compared with ``max``.
- ◆ If ``arr[i] > max``, the current array element becomes the new maximum.
- ◆ After checking all elements, the function returns the largest number.

## Steps in main()

1. Create an array with a maximum size of `100`.
2. Ask the user for the number of elements.
3. Fill the array with random numbers from `1` to `100`.
4. Print all array elements.
5. Set `max` to `0`.
6. Loop through all array elements.
7. Compare each element with `max`.
8. If the current element is greater, update `max`.
9. Continue until all elements have been checked.
10. Return and print the maximum number.

# 💡 Example

◆ Input:

Enter number of elements: 10

◆ Generated array:

42 68 35 1 70 25 79 59 63 65

◆ The program compares the numbers and finds that `79` is the largest value.

➡ Result

◆ The program prints:

Array Elements: 42 68 35 1 70 25 79 59 63 65

Max Number is: 79

## ✈ Note

◆ `max` stores the largest value found so far.

◆ The comparison:

`if (arr[i] > max)`

checks whether the current element is larger than the current maximum.

◆ If the array is `42 68 35 1 70`, the value of `max` changes like this:

`0 → 42 → 68 → 68 → 68 → 70`

◆ Since the generated numbers are always between `1` and `100`, starting `max` at `0` works correctly.

◆ `MaxNumbInArray()` is reusable and can find the maximum of any integer array, as long as the initial value is suitable for the possible data.

## ⊘ Warning

- ◆ `FillArrayWithRandomNumber()` does not validate that `arrLength` is between `1` and `100`.
- ◆ If the user enters more than `100`, the program can access memory outside the array.
- ◆ Starting `max` at `0` would not work correctly if the array could contain only negative numbers. A safer general approach is to initialize `max` with `arr[0]`.
- ◆ The program does not call `srand((unsigned)time(NULL))`, so the random sequence may repeat between executions.

## Comparison

- ▣ `MaxNumbInArray()` → finds the largest element.
- ▣ `PrintArrayElements()` → displays all elements.
- ▣ `FillArrayWithRandomNumber()` → fills the array with random values.
- ▣ `arr[i] > max` → checks whether the current element should become the new maximum.
- ▣ `max` → stores the largest value found so far.

## ✦ Summary

- ✓ The program fills an array with random numbers from `1` to `100`.
- ✓ `PrintArrayElements()` prints the array.
- ✓ `MaxNumbInArray()` searches for the largest value.
- ✓ `max` stores the largest value found so far.
- ✓ Each element is compared with `max`.
- ✓ The array is scanned only once.
- ✓ For the example array, the maximum number is `79`.

Thank you 😊